

Arquitectura y Diseño de una Estación Meteorológica Digital Multi-Agente

Proyecto: Estación Meteorológica Digital con Agente e Inteligencia Artificial (IvekBot Weather Station)
Estándar: Model Context Protocol (MCP), FastMCP, FastAPI, Pydantic v2, Docker
Ubicación de Referencia: Xalapa, Veracruz, México

1. Resumen Ejecutivo y Filosofía del Sistema

El objetivo de este proyecto es construir un ecosistema meteorológico digital autónomo, modular y escalable. A diferencia de las estaciones meteorológicas tradicionales que solo muestran lecturas crudas, este sistema integra **sensores físicos / telemetría en tiempo real** (vía MQTT/HTTP) con **modelos numéricos globales (NWP)** y **agentes de Inteligencia Artificial**.

La arquitectura separa estrictamente la **lógica determinista** (cálculos matemáticos, físicas atmosféricas, conversión de unidades) del **razonamiento cognitivo** (interpretación sinóptica, síntesis de riesgos, generación de alertas públicas).

2. Arquitectura General y Flujo de Datos

flowchart TD

subgraph Sensores & APIs

A1[Estación Física / ESP32 MQTT]

A2[Open-Meteo / ERA5 30y]

A3[GOES-16 Satellite & GLM]

A4[Radar Doppler / Mosaicos]

end

subgraph Orquestador Central

ORQ[Orquestador Master / Router]

end

subgraph Subagentes Especializados

SA1[Subagente 1: Ingesta & Telemetría]

SA2[Subagente 2: Motor Termodinámico & Anomalías]

SA3[Subagente 3: Verificación & Pos-Evento]

SA4[Subagente 4: Alertas, Riesgo & Difusión]

end

subgraph Base de Datos & Auditoría

DB[(PostgreSQL / SQLite)]

AUD[historico_pronosticos/ JSON Logs]

end

A1 --> SA1

A2 --> SA1

A3 --> SA1

A4 --> SA1

SA1 --> ORQ

ORQ <--> SA2

ORQ <--> SA3

ORQ <--> SA4

SA2 --> DB

SA3 --> AUD

SA4 --> PublicOutput[Dashboard / WhatsApp / X / Telegram]

3. Orquestador Central y Jerarquía de Subagentes

3.1. Orquestador Principal (Master Controller & Router)

- **Rol:** Actúa como el centro neurálgico del sistema. Recibe solicitudes de usuarios o triggers programados (cron), evalúa la intención, mantiene el contexto conversacional y delega tareas a los subagentes especializados.
 - **Patrón de Diseño:** *Plan-and-Execute* con validación de esquemas Pydantic.
 - **Funciones Clave:**
 - Enrutamiento inteligente de consultas (e.g., consulta en tiempo real vs. auditoría pos-evento).
 - Ensamblado final de respuestas ejecutivas.
 - Manejo de excepciones y fallbacks cuando una API o sensor físico no responde.
-

3.2. Subagentes Especializados

Subagente 1: Ingesta & Telemetría (Data Ingestion Agent)

- **Responsabilidad:** Conexión y normalización de fuentes de datos heterogéneas.
- **Módulos:**
 - **MQTT/HTTP Broker:** Recoge datos de sensores locales (temperatura, humedad, presión, velocidad/dirección del viento, pluviómetro).
 - **API Fetcher:** Ingesta Open-Meteo, CAMS SAL AOD (polvo del Sahara), NOAA CPC (fases MJO RMM1/RMM2).
 - **Remote Sensing:** Consultas a buckets S3 de GOES-16 (banda 13 IR) y rayado GLM.

Subagente 2: Motor Termodinámico & Anomalías (Thermodynamic Engine Agent)

- **Responsabilidad:** Procesamiento determinista puro. Sin alucinaciones de IA.
- **Cálculos Clave:**
 - **Anomalia Térmica:** $\Delta T = T_{\text{pred}} - \bar{T}_{\text{30y_ERA5}}$.
 - **Índices de Inestabilidad:** CAPE (\$J/kg\$), CIN (\$J/kg\$), Lifted Index (LI), Agua Precipitable (PWAT en \$mm\$).

- **Filtrado de Series Temporales:** Media móvil de 3h para eliminación de ruido estocástico diurno.

Subagente 3: Verificación & Retroalimentación Pos-Evento (Feedback & Calibration Agent)

- **Responsabilidad:** Cierre del bucle de aprendizaje del sistema (24-72h pos-evento).
- **Funciones:**
 - Comparación entre precipitaciones/temperaturas pronosticadas vs. observaciones reales de estaciones automáticas.
 - Cálculo de métricas metrológicas: $\text{MAE} = \frac{1}{n} \sum |Y_t - \hat{Y}_t|$, $\text{RMSE} = \sqrt{\frac{1}{n} \sum (Y_t - \hat{Y}_t)^2}$, $\text{BIAS} = \frac{1}{n} \sum (\hat{Y}_t - Y_t)$
 - Autocalibración dinámica de pesos del ensamble NWE (ECMWF, UKMO, ICON, HRRR).

Subagente 4: Alertas, Riesgo Hidrológico & Difusión (Alerts & Social Agent)

- **Responsabilidad:** Traducción de datos técnicos complejos a lenguaje ciudadano y evaluación de impacto.
- **Módulos:**
 - **Motor de Riesgo por Cuenca:** Monitoreo de umbrales en ríos (Río Actopan, Río La Antigua, Río Jamapa).
 - **Formateador Multicanal:** Generación automática de avisos formateados con emojis para X (Twitter), Facebook, Telegram y WhatsApp.

4. Selección y Benchmarking de Modelos LLM

4.1. Fases de Desarrollo vs. Operación (Runtime)

Rol / Fase	Modelo Recomendado (Opción 1)	Modelo Recomendado (Opción 2)	Criterios de Selección
Desarrollo & Codificación	DeepSeek-V3	Qwen-2.5-Coder-32B	Alta precisión en Python, DRF, Pydantic v2 y FastMCP a una fracción del costo.

Rol / Fase	Modelo Recomendado (Opción 1)	Modelo Recomendado (Opción 2)	Criterios de Selección
Orquestador Principal	DeepSeek-R1	Gemini 1.5 Pro	Razonamiento profundo (CoT), excelente seguimiento de instrucciones complejas y gran ventana de contexto.
Agente del Clima (Runtime Operativo)	Qwen-2.5-72B-Instruct	DeepSeek-V3	Balance perfecto entre costo por token, velocidad de inferencia, llamadas a funciones (Function Calling) y calidad de síntesis.
Subagentes de Extracción Rápidos	Gemini 1.5 Flash	Qwen-2.5-7B-Instruct	Latencia < 500ms, costo casi nulo, ideal para formateo JSON estructurado.

5. Implementación Referencial de Código

5.1. Contratos Pydantic de Auditoría (`schemas.py`)

```
from pydantic import BaseModel, Field
```

```
from typing import List, Optional
```

```
class TelemetryData(BaseModel):
```

```
    station_id: str
```

```
    temperature_c: float
```

```
    humidity_pct: float
```

pressure_hpa: float

rain_rate_mmh: float

wind_speed_kmh: float

class ThermodynamicIndices(BaseModel):

cape_jkg: float = Field(..., description="Convective Available Potential Energy")

cin_jkg: float = Field(..., description="Convective Inhibition")

lifted_index: float

pwat_mm: float = Field(..., description="Precipitable Water")

class ForecastReport(BaseModel):

timestamp: str

location: str

telemetry: Optional[TelemetryData] = None

thermodynamics: ThermodynamicIndices

summary_text: str

alert_level: str

5.2. Código del Orquestador (`orchestrator.py`)

import httpx

from fastapi import FastAPI

from pydantic import BaseModel

app = FastAPI(title="IvekBot Weather Orchestrator")

@app.post("/api/v1/forecast/run")

```
async def run_weather_pipeline(location: str = "Xalapa, Veracruz"):
```

```
    # 1. Ingesta
```

```
    telemetry = await fetch_telemetry(location)
```

```
    # 2. Motor determinista
```

```
    thermo = calculate_thermodynamics(telemetry)
```

```
    # 3. Verificación
```

```
    feedback = evaluate_post_event(location)
```

```
    # 4. Generación de Alerta
```

```
    report = generate_alert(telemetry, thermo, feedback)
```

```
    return {
```

```
        "status": "SUCCESS",
```

```
        "data": report
```

```
    }
```

6. Despliegue y Arquitectura Docker

`docker-compose.yml`

```
version: "3.8"
```

```
services:
```

```
    orchestrator:
```

```
        build: .
```

ports:

- "8000:8000"

environment:

- DATABASE_URL=postgresql://user:password@db:5432/weather_db
- OPEN_METEO_API=https://api.open-meteo.com/v1

depends_on:

- db
- mqtt_broker

mqtt_broker:

image: eclipse-mosquitto:latest

ports:

- "1883:1883"

db:

image: postgres:15-alpine

environment:

POSTGRES_USER: user

POSTGRES_PASSWORD: password

POSTGRES_DB: weather_db

volumes:

- postgres_data:/var/lib/postgresql/data

volumes:

postgres_data:

